Project Report: Messy Mashup Genre Classification

Competition: Jan 2026 DLGenAI Project

Name: Kartik Bajaj | **Roll No:** 24F2007692 | **Date:** April 2, 2026

1. Abstract

This report details our strategy for the Messy Mashup Kaggle competition, where the goal is to classify genres from noisy mashups. The core challenge is the **distribution gap**: training data has clean stems, while test data contains noisy, mixed audio. We implemented multiple models, from classical ML to a custom CNN and advanced pre-trained models (ResNet-50 and AST). Our final submission, a weighted ensemble of AST and ResNet-50, achieved a **Kaggle Public Score of 0.954**, safely above the 0.80 cutoff.

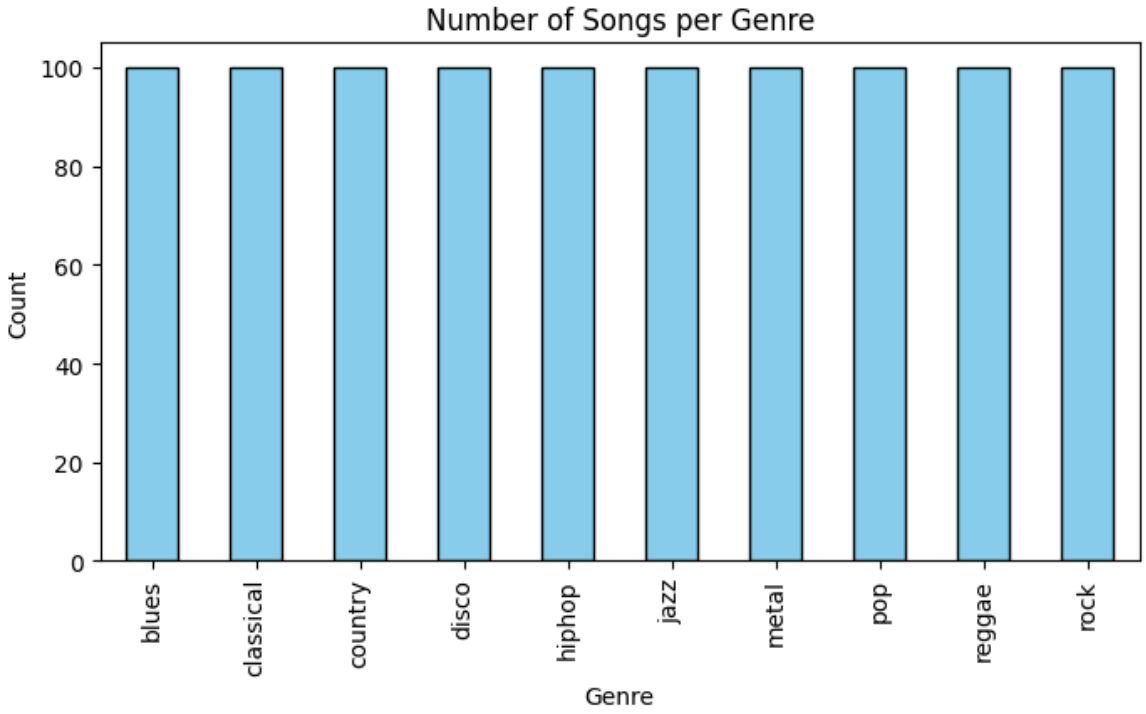
2. Problem Statement

The Challenge: We must predict 10 music genres from audio. Training data consists of clean, separated stems (vocals, drums, bass). However, the test data features mixed mashups with tempo adjustments and added environmental noise (ESC-50).
Objectives:

- Overcome the clean-to-noisy gap.
- Build at least 3 unique architectures (one scratch, one pre-trained, one choice).
- Prevent data leakage to ensure trustworthy metrics (Macro F1).

3. Dataset and Preprocessing

Training Data: 1,000 clean songs evenly split across 10 genres (perfectly balanced). Each has 4 stems.



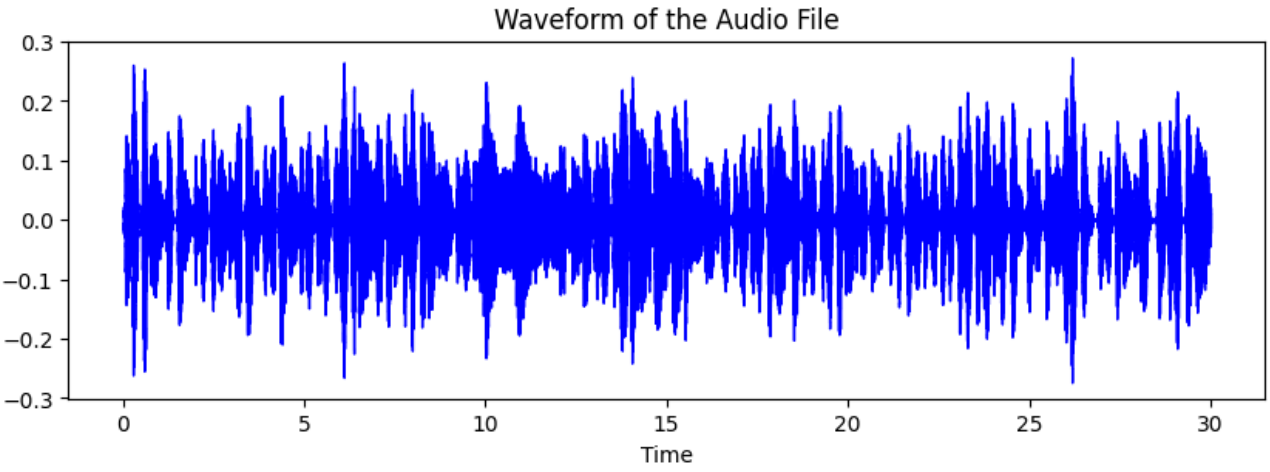
Test Data: 3,020 noisy mashups involving tempo variations and random background noises.

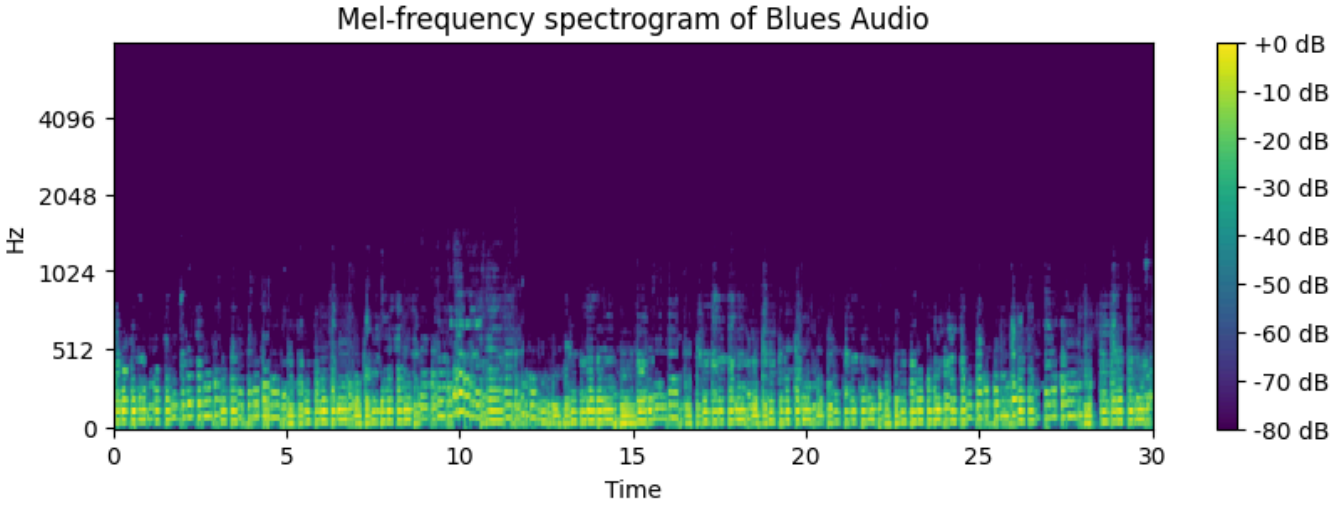
Our Solution (Synthetic Mashups): Because of the gap, we generated custom training mashups. We mixed stems from different songs inside the same genre and added random noise. We generated 10,000 synthetic audio files to teach our models how to handle noisy environments.

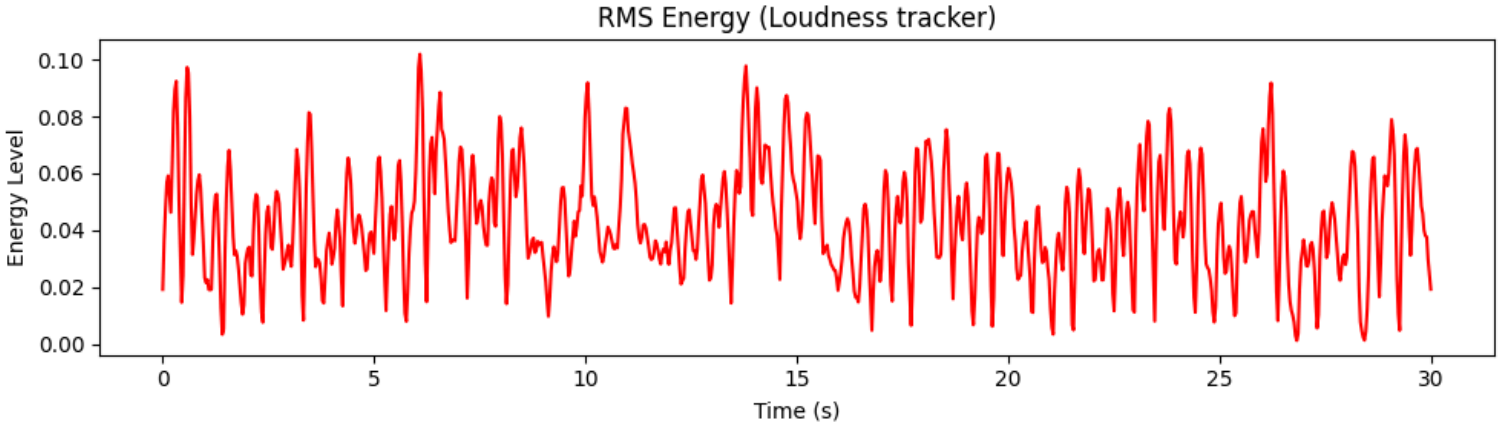
Data Leakage Prevention: We used **StratifiedGroupKFold** grouped by original song. This ensures pieces of the same track did not bleed into validation folds.

4. Feature Extraction

1. **MFCCs:** Used for baseline ML models. Averaging them ignored musical timing, leading to poor results.
2. **Mel-Spectrograms:** Used for all Deep Learning models. By converting audio into 2D heatmap images, our models learned both rhythm (time) and music frequencies (pitch), drastically improving performance.







5. Modeling and Experimentation

We tracked 31 experiments using Weights & Biases.

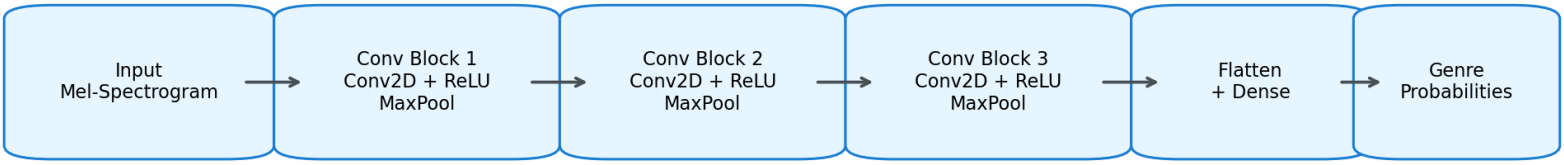
5.1 Classical Baselines

- **Random Forest & Gradient Boosting:** Struggled using flat MFCCs. Peak Macro F1: **0.4954 (RF)**.
- Both were severely limited by lack of spatial memory.

5.2 AudioCNN (Built from Scratch)

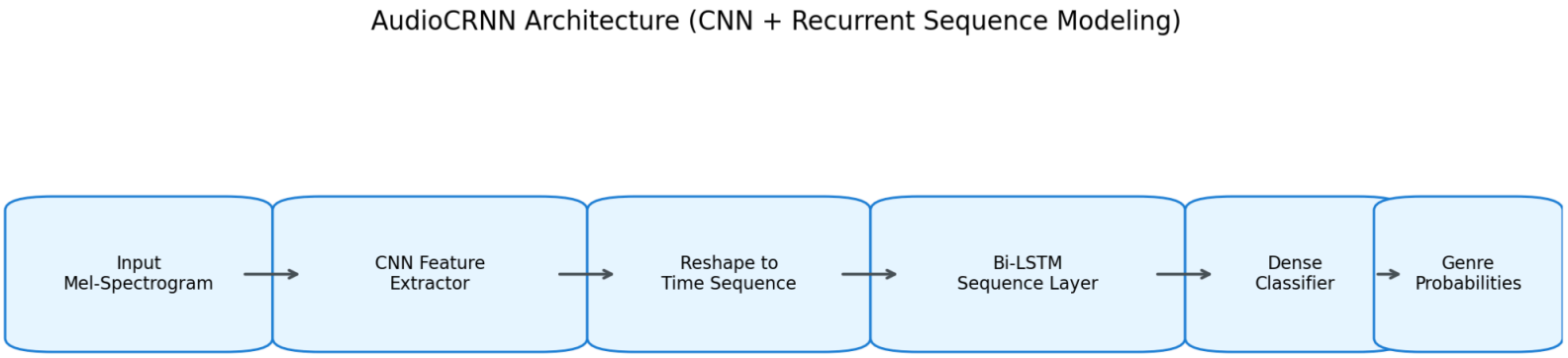
- **Design:** 3 Convolutional layers (1→16, 16→32, 32→64) → Max Pooling → Adaptive Pooling → Fully Connected.
- **Result: 0.6867 Macro F1**. A massive jump from baselines, proving 2D spectrograms and CNNs work beautifully together for audio.

Custom AudioCNN Architecture (Project Pipeline)



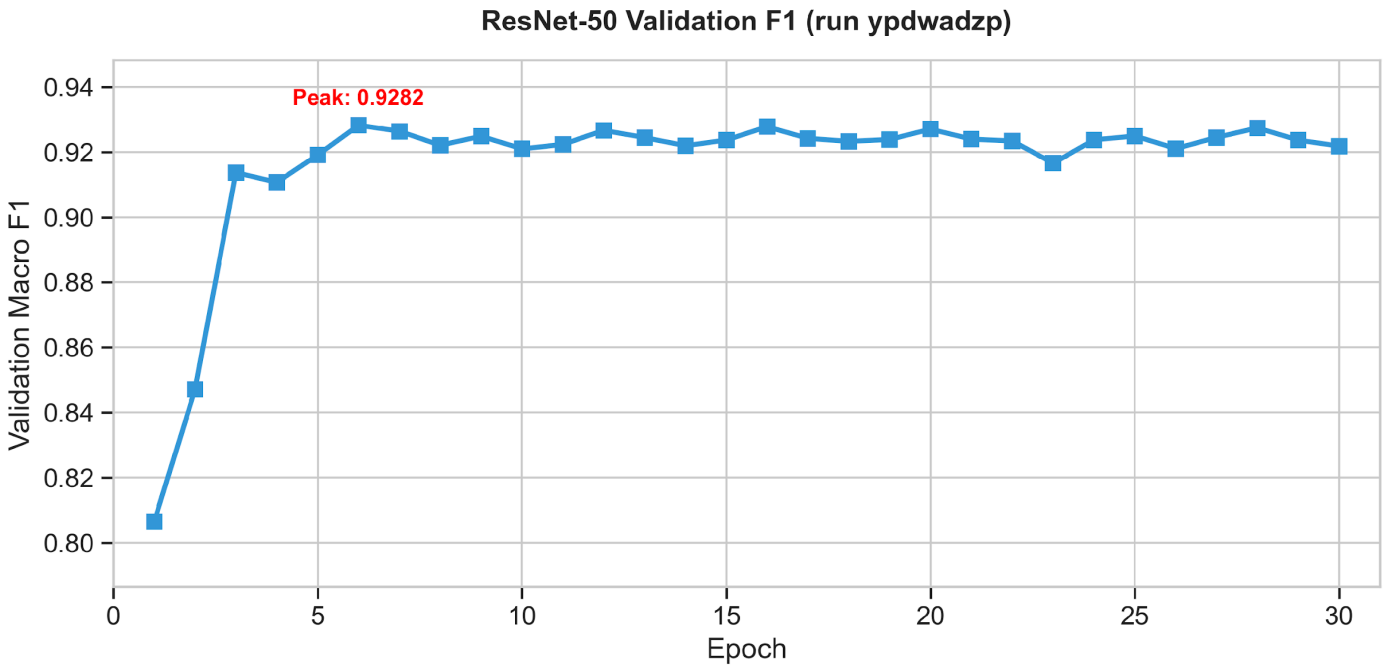
5.3 AudioCRNN (CNN + LSTM)

- **Design:** CNN encoder fed into a 2-layer LSTM to capture long-term context.
- **Result: 0.4818 Macro F1**. Underperformed because the LSTM added far too many parameters. Our dataset was simply too small to train recurrent networks effectively without heavier augmentation.



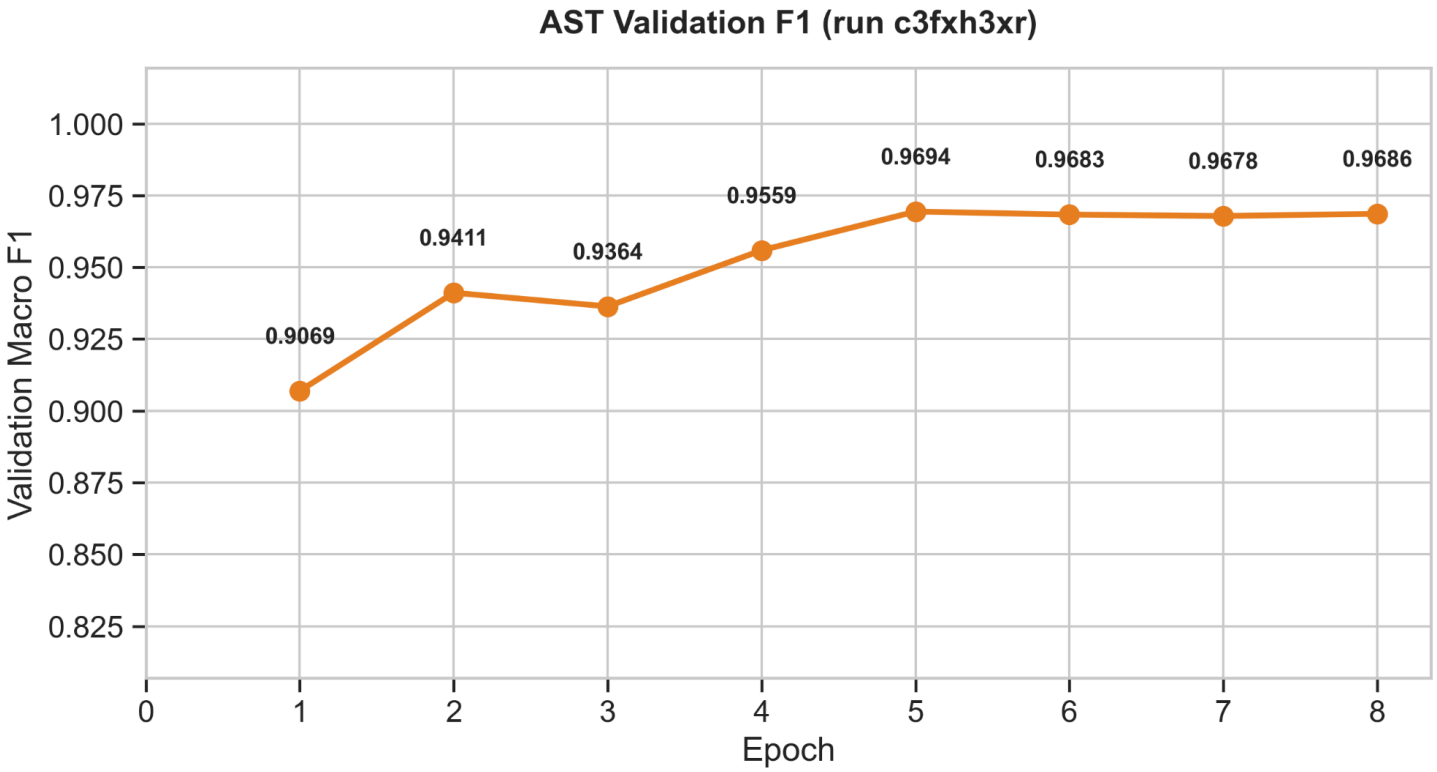
5.4 ResNet-50 (Transfer Learning)

- **Design:** Replaced the final layer of a pre-trained ResNet-50 model.
- **Result:** Peak Validation F1 = **0.9282**. Transfer learning provided an enormous boost (+0.24). The pre-trained ImageNet weights easily extracted visual patterns mapping to specific musical traits.



5.5 Audio Spectrogram Transformer (AST)

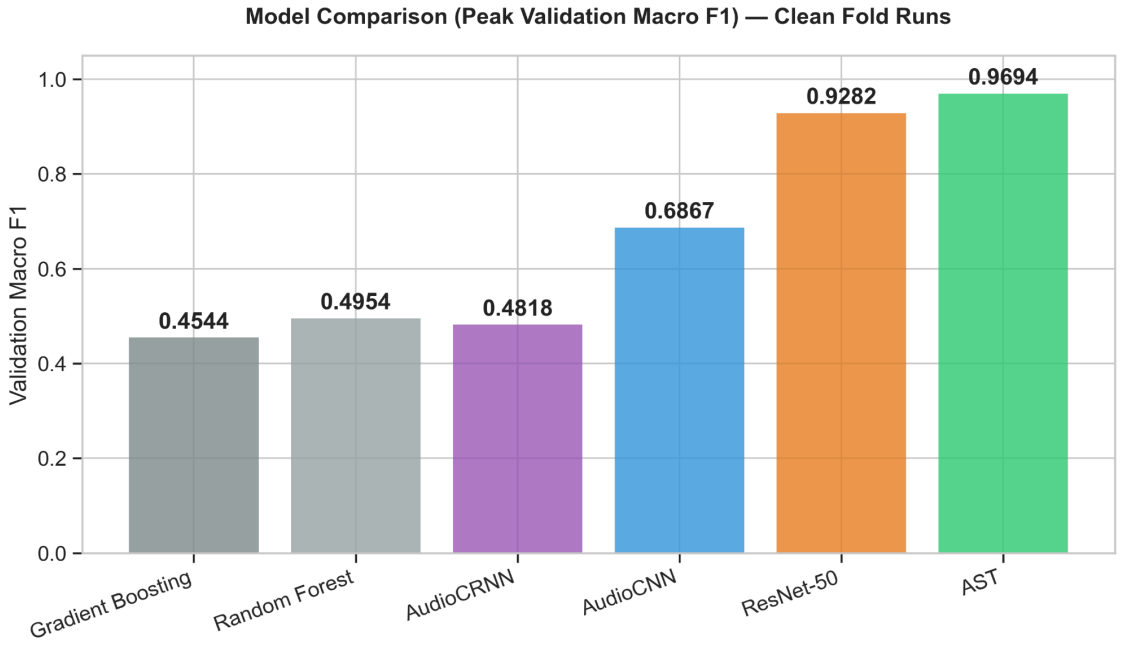
- **Design:** Pre-trained specifically on AudioSet, we fine-tuned the classification head.
- **Result:** Peak Validation F1 = **0.9694**. AST outperformed everything.
- Because of its self-attention mechanism, it looks at the entire audio sequence simultaneously to find genre cues anywhere within the song window.

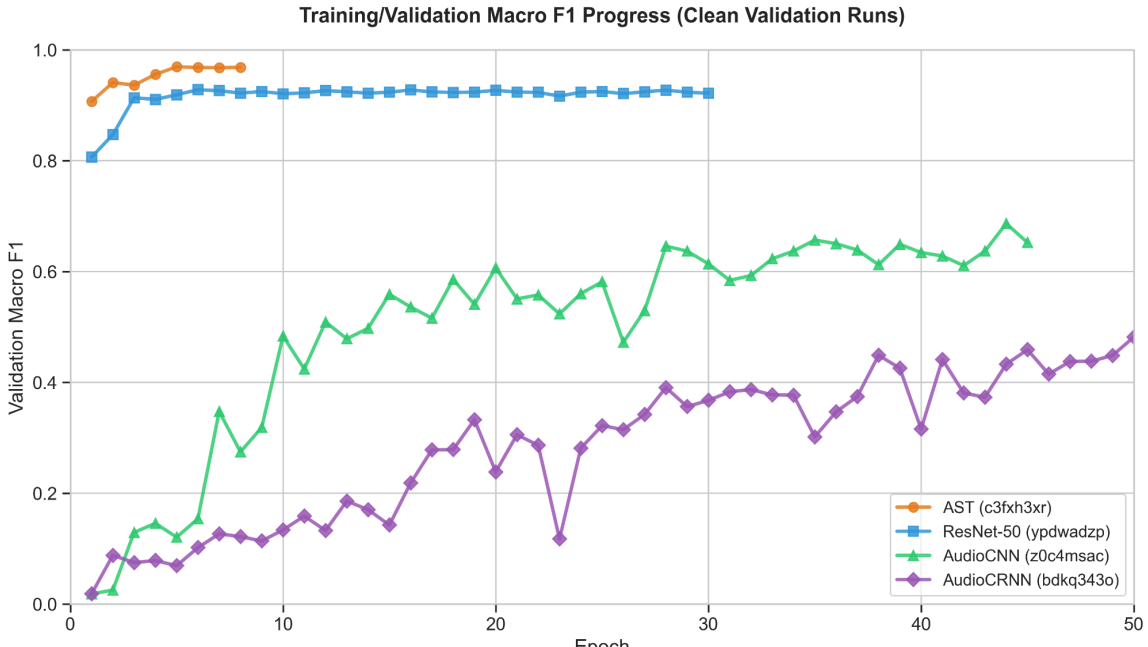


6. Final Results and Ensemble

Below is the summary of peak performance across cleanly validated, non-leaked fold runs.

Model	Type	Val Macro F1	Kaggle score
Gradient Boosting	Baseline	0.4544	0.40
Random Forest	Baseline	0.4954	0.40
AudioCRNN	CNN + LSTM	0.4818	0.42
AudioCNN	Scratch CNN	0.6867	0.50
ResNet-50	Pre-Trained CNN	0.9282	0.90
AST	Pre-Trained Transformer	0.9694	0.93





Final Submission Pipeline: To maximize test accuracy, we processed predictions using **Test-Time Augmentation (TTA)** for our top two models. We ran 4 augmentations per file. We then combined their probabilities in a weighted ensemble: **60% AST + 40% ResNet-50**.

Final Kaggle Leaderboard Score: 0.954

7. Model Deployment

To make the **project interactive** and easy to **test**, the final trained **AST model** was deployed as a live web application.

How it was built:

- **Web UI:** I used Python's Gradio library to build a simple interface where users can either upload an audio file (.wav, .mp3) or record directly from their microphone.
- **Hosting:** The application is hosted on Hugging Face Spaces, which provides a reliable server environment for PyTorch models.
- **Background Process:** When a user submits audio, the app uses librosa to format the audio to 16kHz. It then passes the audio through the ASTFeatureExtractor and feeds it to the trained model to instantly predict the audio's genre. To keep the app fast on a basic CPU, the prediction step is wrapped in torch.no_grad() to save memory.

Project Links:

Live Web App: Audio-Classifier-genai

Source Code: dl-genai-project-26-t1

Status: Live and successfully classifying audio in real-time.

8. Conclusions

1. **Data Augmentation bridged the gap:** Creating synthetic mashups solved the difference between clean training data and noisy test data.
2. **Spectrograms > MFCCs:** Viewing audio as pictures let deep learning models find rich structural patterns.
3. **Transfer Learning is Dominant:** Heavy pre-trained models (AST, ResNet) absolutely crushed models trained from scratch.
4. **Ensembling gives stability:** Blending AST and ResNet smoothed out boundary errors between overlapping genres like Rock and Metal.

9. References

1. Gong, Y., et al. (2021). *AST: Audio Spectrogram Transformer*. INTERSPEECH 2021.
2. He, K., et al. (2016). *Deep Residual Learning for Image Recognition*. CVPR 2016.
3. McFee, B., et al. (2015). *librosa: Audio and Music Signal Analysis in Python*. SciPy.
4. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS.
5. Weights & Biases Documentation. https://wandb.ai/
6. Wolf, T., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. EMNLP 2020.
7. Piczak, K. J. (2015). *ESC: Dataset for Environmental Sound Classification*. ACM Multimedia.